

Procesos Estocásticos - Tarea Examen

Ruben Gascon Calixto

2026-06-24

Contents

Ejercicio 1a: Caminata Aleatoria y Martingala	2
Descripción	2
Funciones	2
Simulación y Visualización	2
Ejercicio 1b: Procesos Reescalados	3
Descripción	3
Funciones	4
Simulación y Visualización	4
Ejercicio 1c: Procesos de Precios	5
Descripción	5
Funciones	5
Simulación y Visualización	6
Ejercicio 1d: Análisis Completo con Múltiples Configuraciones	7
Descripción	7
Funciones	7
Simulaciones para Diferentes N	8
Simulación de Monte Carlo para Valoración de Opción Call Europea	11
Parámetros de la Opción	11
Simulación de Monte Carlo	11
Fórmula de Black-Scholes	12
Comparación de Métodos	12
Análisis de la Simulación	12

Ejercicio 1a: Caminata Aleatoria y Martingala

Descripción

Generamos una caminata aleatoria que se convierte en martingala mediante el ajuste apropiado de la probabilidad p^* .

Funciones

```
# Calcular p* para garantizar propiedad de martingala
calcular_p_estrella <- function(a = 1.1, b = -0.1, alpha = 0.04) {
  numerador <- -(a - b)^2 + sqrt((a - b)^4 - 4 * (a - b)^2 * 2 * (exp(alpha) - 1))
  denominador <- -2 * (a - b)^2
  return(numerador / denominador)
}

# Generar caminata aleatoria
generar_caminata_aleatoria <- function(N = 100, a = 1.1, b = -0.1, alpha = 0.04) {
  p_estrella <- calcular_p_estrella(a, b, alpha)
  cat(sprintf("Valor de p* calculado: %.4f\n", p_estrella))

  # Generar pasos aleatorios
  X <- sample(c(a, b), size = N, replace = TRUE, prob = c(p_estrella, 1 - p_estrella))

  # Caminata acumulativa
  S_k <- c(0, cumsum(X))

  # Ajuste para martingala
  f <- function(k) k * (p_estrella * a + (1 - p_estrella) * b)
  Z_k <- S_k - sapply(0:N, f)

  return(list(S_k = S_k, Z_k = Z_k, p_estrella = p_estrella))
}
```

Simulación y Visualización

```
N <- 100
resultado <- generar_caminata_aleatoria(N)
```

```
## Valor de p* calculado: 0.0603
```

```
# Crear dataframe para graficar
df_plot <- data.frame(
  k = 0:N,
  S_k = resultado$S_k,
  Z_k = resultado$Z_k
)

# Gráfico de S_k
```

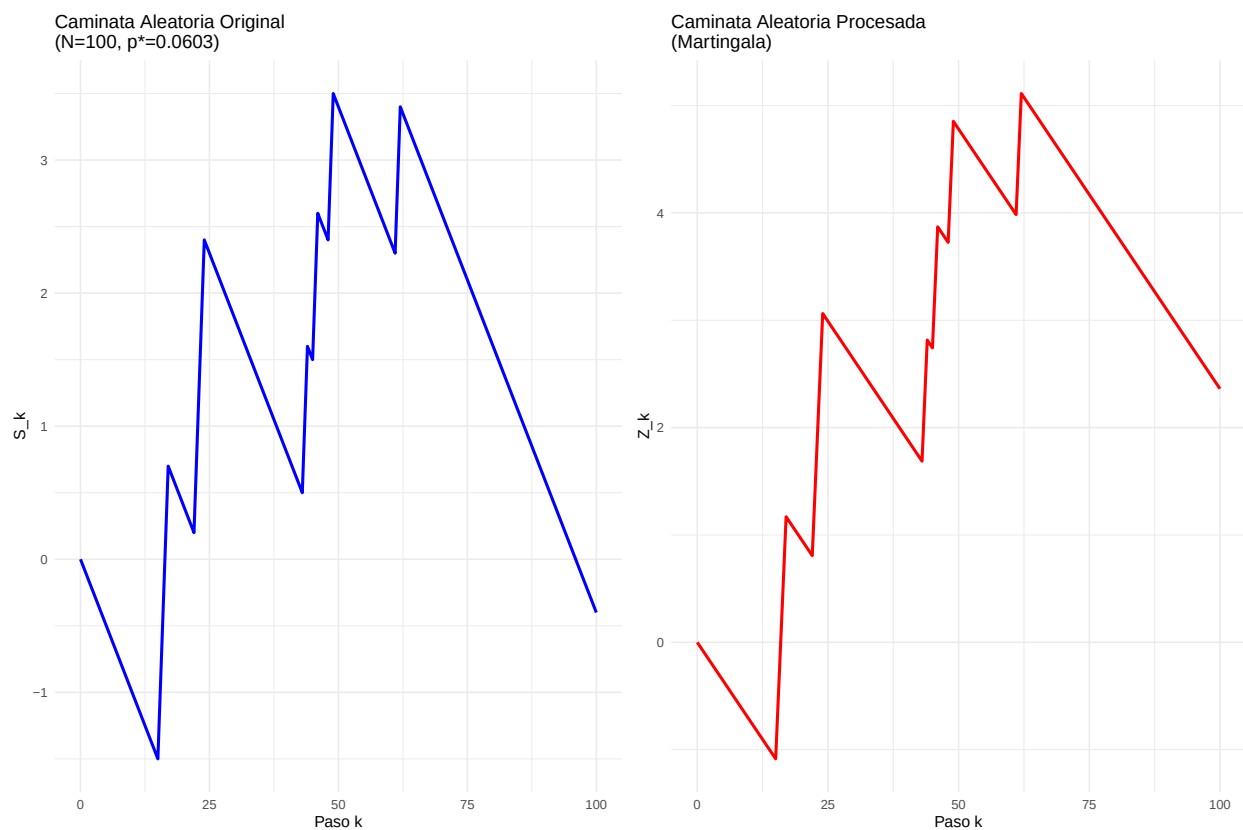
```

p1 <- ggplot(df_plot, aes(x = k, y = S_k)) +
  geom_line(color = "blue", linewidth = 1) +
  labs(title = sprintf("Caminata Aleatoria Original\n(N=%d, p*=%.4f)", N, resultado$p_estrella),
       x = "Paso k", y = "S_k") +
  theme_minimal()

# Gráfico de Z_k
p2 <- ggplot(df_plot, aes(x = k, y = Z_k)) +
  geom_line(color = "red", linewidth = 1) +
  labs(title = "Caminata Aleatoria Procesada\n(Martingala)",
       x = "Paso k", y = "Z_k") +
  theme_minimal()

grid.arrange(p1, p2, ncol = 2)

```



Ejercicio 1b: Procesos Reescalados

Descripción

Generamos procesos reescalados $W^{(r)}$ para diferentes valores de r y los interpolamos linealmente.

Funciones

```
# Calcular p* con parámetro r
calcular_p_estrella_r <- function(a, b, alpha, r) {
  delta <- a - b
  numerador <- -delta^2 + sqrt(delta^4 - 4 * delta^2 * 2 * ((1 + alpha/r)^r - 1))
  denominador <- -2 * delta^2
  return(numerador / denominador)
}

# Generar proceso reescalado
generar_proceso_reescalado <- function(N, a, b, alpha, r) {
  p_estrella <- calcular_p_estrella_r(a, b, alpha, r)

  # Generar caminata
  X <- sample(c(a, b), size = N, replace = TRUE, prob = c(p_estrella, 1 - p_estrella))
  S <- c(0, cumsum(X))

  # Reescalar
  tiempos <- seq(0, N/r, length.out = length(S))
  W_r <- S / sqrt(r)

  return(data.frame(t = tiempos, W_r = W_r, r = r))
}
```

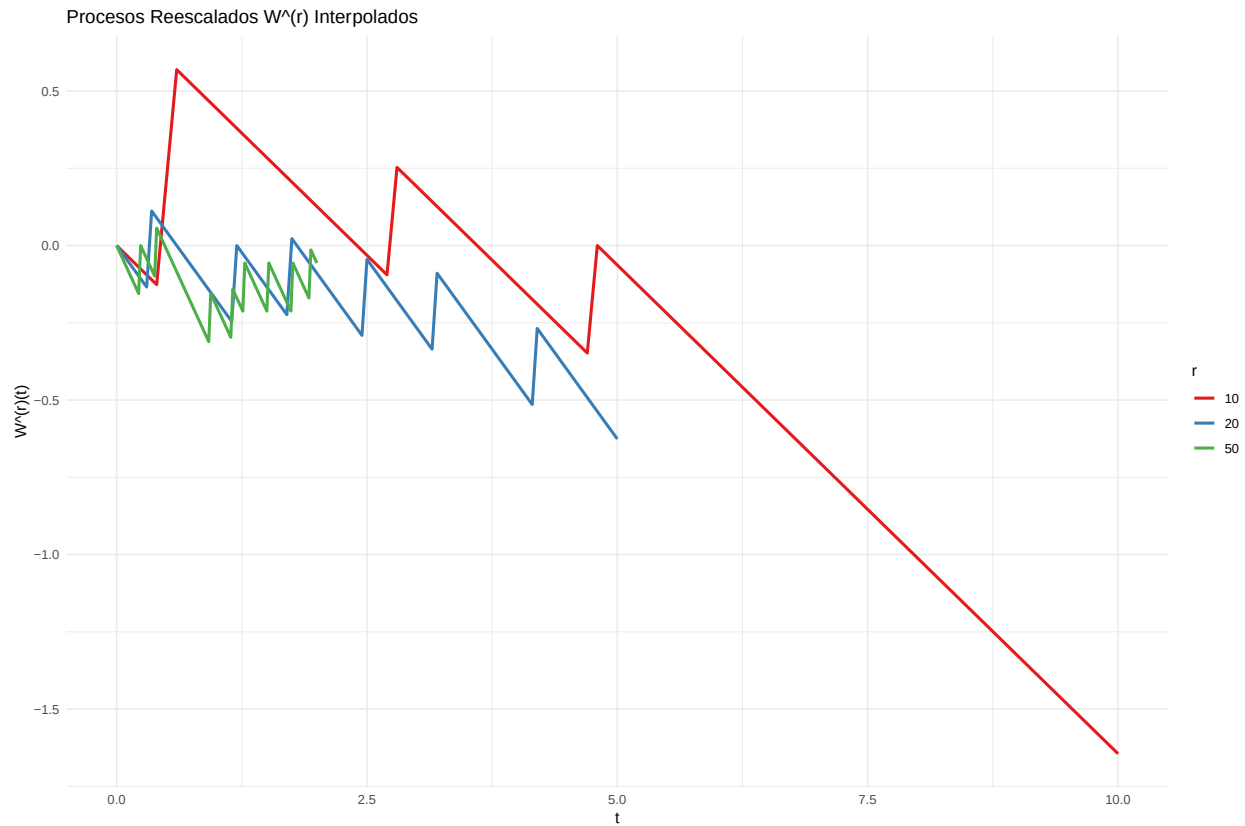
Simulación y Visualización

```
a <- 1.1
b <- -0.1
alpha <- 0.04
N <- 100
valores_r <- c(10, 20, 50)

# Generar procesos para diferentes r
df_procesos <- do.call(rbind, lapply(valores_r, function(r) {
  cat(sprintf("r = %d, P* = %.7f\n", r, calcular_p_estrella_r(a, b, alpha, r)))
  generar_proceso_reescalado(N, a, b, alpha, r)
})))
```

```
## r = 10, P* = 0.0601890
## r = 20, P* = 0.0602545
## r = 50, P* = 0.0602939
```

```
# Graficar
ggplot(df_procesos, aes(x = t, y = W_r, color = factor(r))) +
  geom_line(linewidth = 1) +
  labs(title = "Procesos Reescalados W^(r) Interpolados",
       x = "t", y = "W^(r)(t)", color = "r") +
  theme_minimal() +
  scale_color_brewer(palette = "Set1")
```



Ejercicio 1c: Procesos de Precios

Descripción

Modelamos la evolución de precios usando el proceso reescalado con la fórmula exponencial.

Funciones

```
# Generar proceso de precios
generar_proceso_precios <- function(N, a, b, alpha, r) {
  p_estrella <- calcular_p_estrella_r(a, b, alpha, r)

  # Generar caminata
  X <- sample(c(a, b), size = N, replace = TRUE, prob = c(p_estrella, 1 - p_estrella))
  S <- c(0, cumsum(X))

  # Escala de tiempo
  t <- seq(0, 1, length.out = length(S))
  W_r <- S / sqrt(r)

  # Calcular precios
```

```

precios <- 110 * exp(-alpha * t + W_r)

return(data.frame(t = t, precio = precios, r = r))
}

```

Simulación y Visualización

```

valores_r <- c(10, 20, 50)

# Generar procesos de precios
df_precios <- do.call(rbind, lapply(valores_r, function(r) {
  cat(sprintf("r = %d, P* = %.7f\n", r, calcular_p_estrella_r(a, b, alpha, r)))
  generar_proceso_precios(N, a, b, alpha, r)
}))

```

```

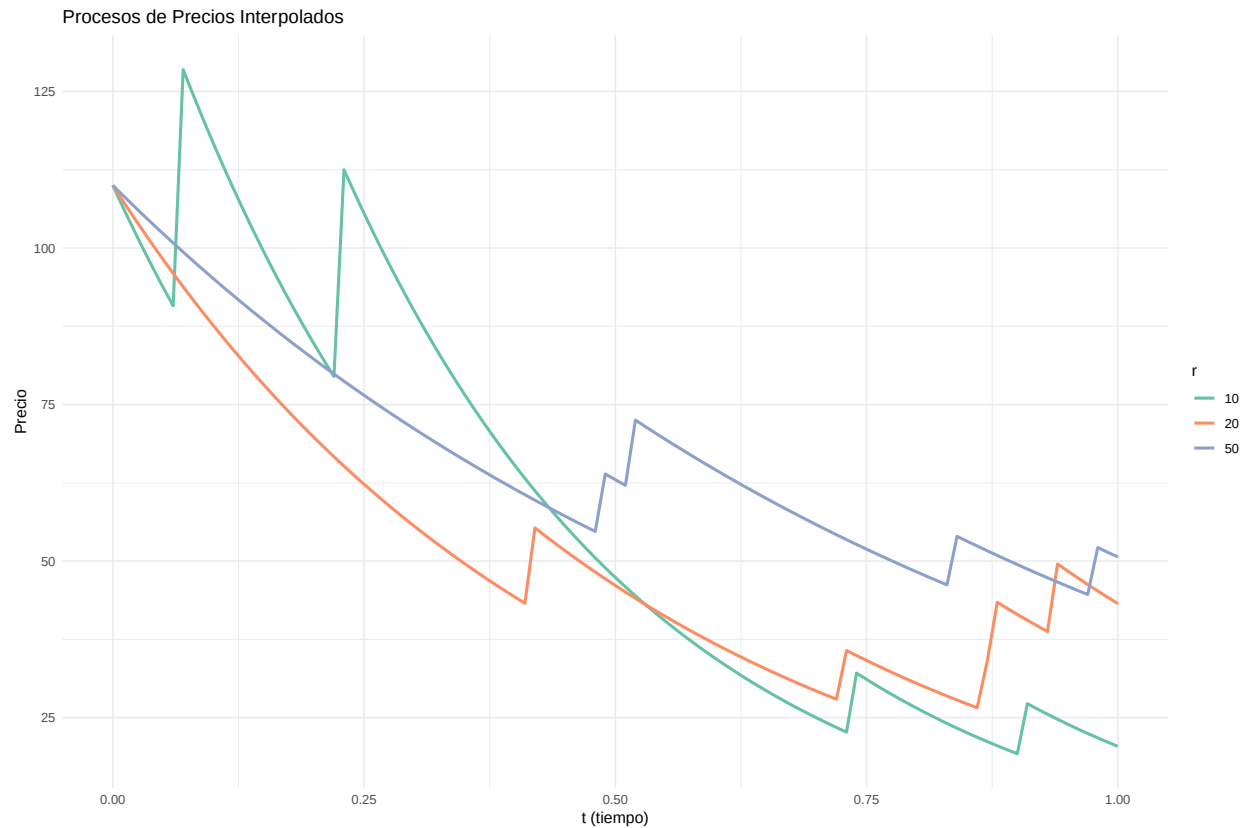
## r = 10, P* = 0.0601890
## r = 20, P* = 0.0602545
## r = 50, P* = 0.0602939

```

```

# Graficar
ggplot(df_precios, aes(x = t, y = precio, color = factor(r))) +
  geom_line(linewidth = 1) +
  labs(title = "Procesos de Precios Interpolados",
       x = "t (tiempo)", y = "Precio", color = "r") +
  theme_minimal() +
  scale_color_brewer(palette = "Set2")

```



Ejercicio 1d: Análisis Completo con Múltiples Configuraciones

Descripción

Realizamos un análisis exhaustivo con diferentes tamaños de simulación y valores de reescalado.

Funciones

```
# Generar caminata aleatoria completa
generar_caminata_completa <- function(N, p_estrella, a = 1.1, b = -0.1) {
  X <- sample(c(a, b), size = N, replace = TRUE, prob = c(p_estrella, 1 - p_estrella))
  S <- c(0, cumsum(X))

  # Ajuste martingala
  k <- 0:N
  f_k <- k * (p_estrella * a + (1 - p_estrella) * b)
  Z <- S - f_k

  return(list(S = S, Z = Z))
}
```

```

# Generar proceso geométrico
generar_proceso_geometrico <- function(Z, r, alpha, S0 = 110) {
  k <- length(Z) - 1
  tiempos <- seq(0, k/r, by = 1/r)
  indices <- pmin(as.integer(tiempos * r) + 1, length(Z))

  W_r <- Z[indices] / sqrt(r)
  proceso <- S0 * exp(-alpha * tiempos + W_r)

  return(data.frame(t = tiempos, precio = proceso))
}

```

Simulaciones para Diferentes N

```

a <- 1.1
b <- -0.1
alpha <- 0.04
S0 <- 110

# Configuraciones
configs <- list(
  list(N = 500, rs = c(50, 100, 250)),
  list(N = 1000, rs = c(100, 200, 500)),
  list(N = 5000, rs = c(500, 1000, 2500))
)

# Procesar cada configuración
for (config in configs) {
  N <- config$N
  rs <- config$rs

  cat(sprintf("\n=== Simulación para N = %d ===\n", N))

  # Generar datos para cada r
  df_completo <- do.call(rbind, lapply(rs, function(r) {
    p_estrella <- calcular_p_estrella_r(a, b, alpha, r)
    cat(sprintf("  r = %d, P* = %.7f\n", r, p_estrella))

    resultado <- generar_caminata_completa(N, p_estrella, a, b)
    df_geo <- generar_proceso_geometrico(resultado$Z, r, alpha, S0)
    df_geo$r <- r
    df_geo
  })))

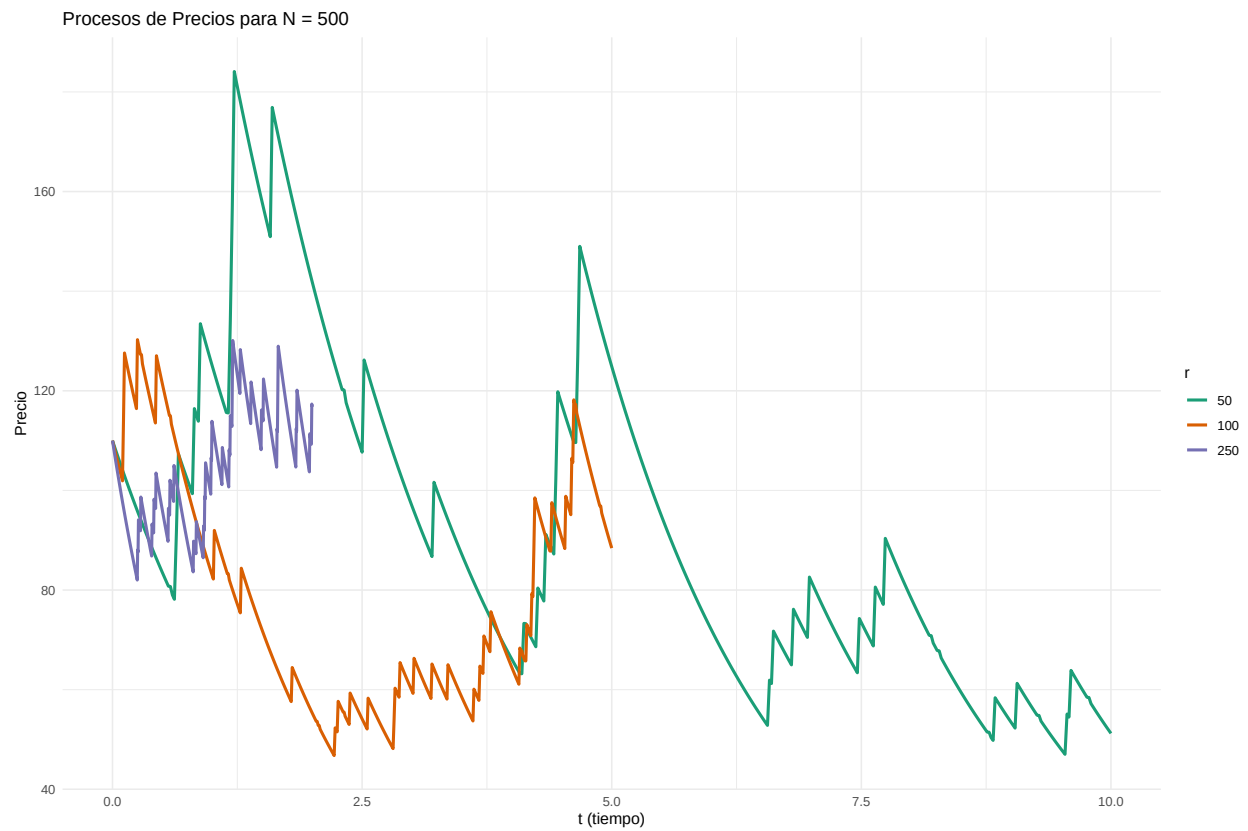
  # Graficar
  p <- ggplot(df_completo, aes(x = t, y = precio, color = factor(r))) +
    geom_line(linewidth = 1) +
    labs(title = sprintf("Procesos de Precios para N = %d", N),
         x = "t (tiempo)", y = "Precio", color = "r") +
    theme_minimal() +
    scale_color_brewer(palette = "Dark2")
}

```

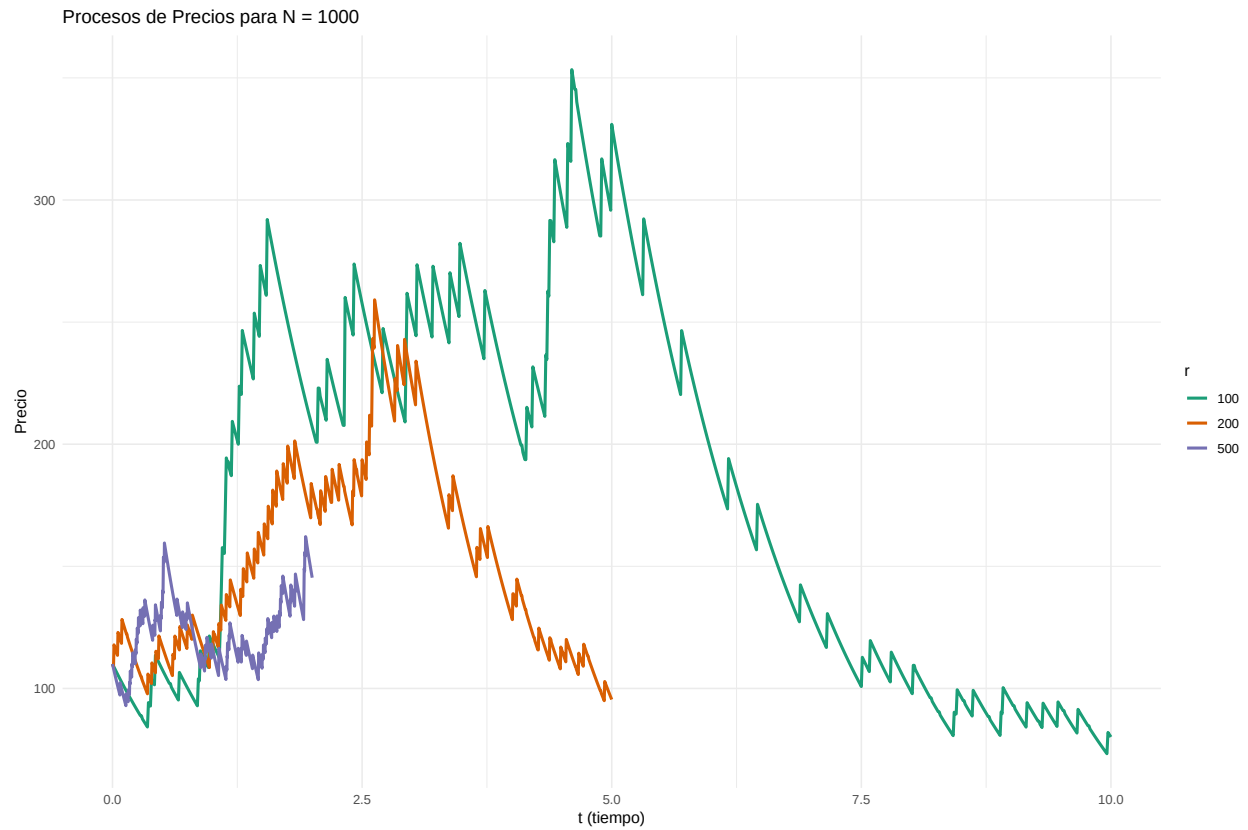


```
print(p)
}
```

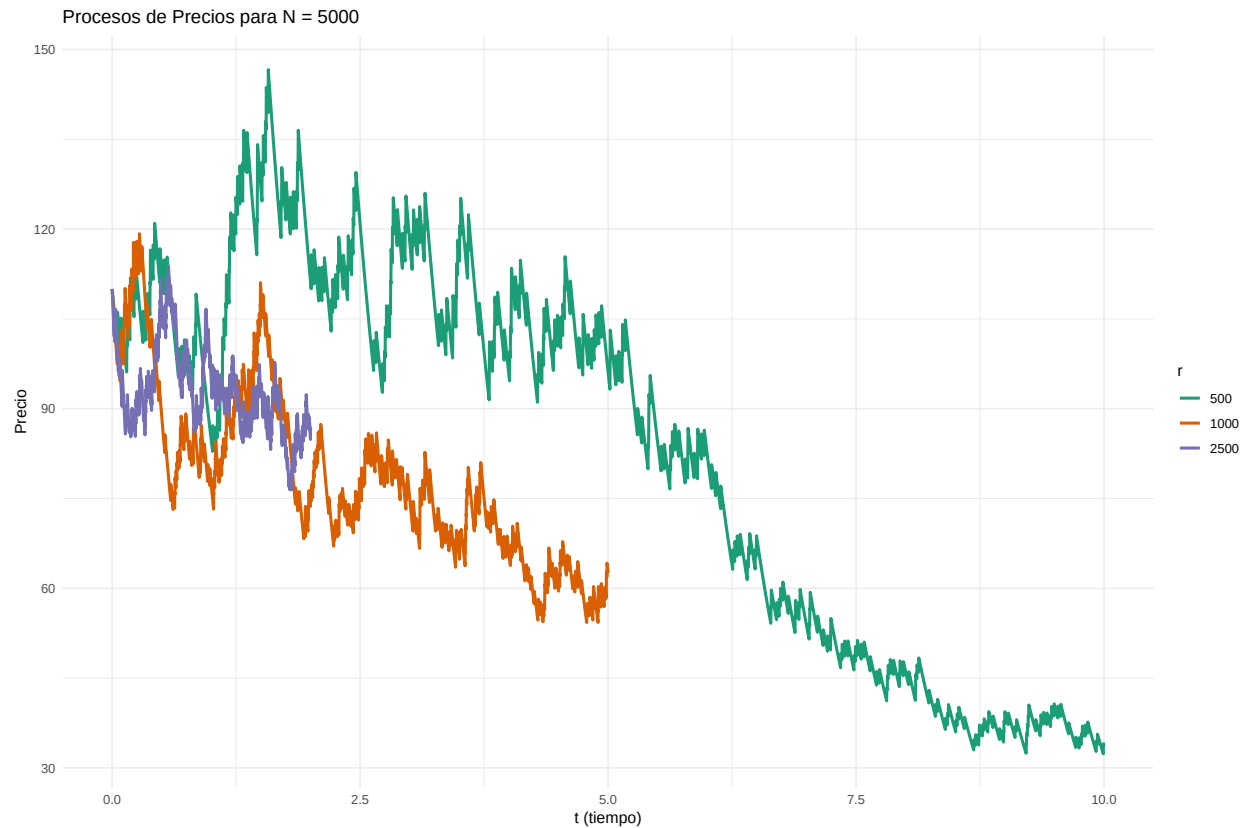
```
##
## === Simulación para N = 500 ===
##   r = 50, P* = 0.0602939
##   r = 100, P* = 0.0603070
##   r = 250, P* = 0.0603149
```



```
##
## === Simulación para N = 1000 ===
##   r = 100, P* = 0.0603070
##   r = 200, P* = 0.0603136
##   r = 500, P* = 0.0603175
```



```
##  
## === Simulación para N = 5000 ===  
##   r = 500, P* = 0.0603175  
##   r = 1000, P* = 0.0603188  
##   r = 2500, P* = 0.0603196
```



Simulación de Monte Carlo para Valoración de Opción Call Europea

Parámetros de la Opción

```
# Establecer semilla para reproducibilidad
set.seed(42)

# Parámetros de la opción
alpha <- 0.04 # Tasa libre de riesgo
S0 <- 110     # Precio inicial del activo
K <- 113      # Precio de ejercicio
N <- 100000   # Número de simulaciones
```

Simulación de Monte Carlo

```
# Simular variables aleatorias X1 siguiendo distribución lognormal
muestras_X1 <- rlnorm(N, meanlog = 0, sdlog = sqrt(2 * alpha))
```

```
# Calcular el payoff descontado para cada simulación
valores_payoff <- pmax(exp(-alpha) * (S0 * muestras_X1 - K), 0)

# Estimar el valor de la opción como el promedio de los payoffs
valor_montecarlo <- mean(valores_payoff)
```

Resultado Monte Carlo: 12.9987

Fórmula de Black-Scholes

```
# Parámetros de la opción para Black-Scholes
S0_bs <- 110 # Precio inicial del activo
K_bs <- 113 # Precio de ejercicio
r_bs <- 0.04 # Tasa libre de riesgo
T_bs <- 1 # Tiempo hasta expiración
sigma_bs <- sqrt(2 * 0.04) # Volatilidad

# Cálculo de parámetros d1 y d2
d1 <- (log(S0_bs/K_bs) + (r_bs + 0.5 * sigma_bs^2) * T_bs) / (sigma_bs * sqrt(T_bs))
d2 <- d1 - sigma_bs * sqrt(T_bs)

# Cálculo del precio de la opción call
call_price <- S0_bs * pnorm(d1) - K_bs * exp(-r_bs * T_bs) * pnorm(d2)
```

Resultado Black-Scholes: 13.0189

Comparación de Métodos

```
diferencia <- abs(valor_montecarlo - call_price)
error_relativo <- (diferencia / call_price) * 100
```

Resumen de Resultados

Método	Valor de la Opción
Monte Carlo	12.9987
Black-Scholes	13.0189
Diferencia	0.0202
Error Relativo	0.16%

Análisis de la Simulación

Estadísticas de la simulación Monte Carlo:

- Desviación estándar: 21.9631
- Error estándar: 0.0695

- Intervalo de confianza (95%): $[12.8626, 13.1348]$

La simulación de Monte Carlo proporciona una aproximación al valor teórico de Black-Scholes, con un error relativo del 0.16%.